

Bilinear Image Scaler in Verilog

A four-stage streaming hardware pipeline that resizes images with bilinear interpolation — AXI-Stream in, AXI-Stream out, no frame-grabber magic.

Why this exists

Many image-scaler implementations built as class projects or tutorials take the simplest path: load the entire picture into a single array and index into it freely, since a testbench allows that. Real hardware does not offer the same luxury — a design cannot afford to hold a full 1080p frame in on-chip memory just to look up an arbitrary pixel on demand. This project was built to respect that constraint directly: pixels arrive one at a time over a stream, in raster order, and the scaler keeps only as much of the image as it needs to produce correct output, discarding the rest as it goes.

So this project is a bilinear image scaler, written in plain Verilog, that resizes an image (up or down, and independently in X and Y) as it streams through, using a small rolling cache of source rows instead of a full frame buffer. It speaks AXI-Stream on both ends so it can drop into a larger video pipeline without anyone needing to know what's inside it.

What it's actually doing

Bilinear scaling means every output pixel is a blend of its four nearest neighbours in the source image — top-left, top-right, bottom-left, bottom-right — weighted by how close the output pixel's mapped position sits to each of them. It's the same idea Photoshop, GIMP, or your phone's camera app use when you resize something and want it to look smooth instead of blocky.

The math is simple, but doing it as a hardware pipeline instead of a for-loop means every step above has to be broken into its own pipeline stage, driven by a clock, and kept correctly synchronized with everything around it — which is where basically all of the actual engineering effort in this repo went.

The pipeline, stage by stage

scaler_top wires together four stages, in order: the coordinate mapper, the row buffer, the horizontal interpolator, and the vertical interpolator. Each stage talks to its neighbours with a plain valid/ready handshake, so any stage can stall the ones behind it without losing data — the same backpressure discipline AXI-Stream uses at the edges of the design.

Stage 1 — mapper.v

For every destination pixel, this stage figures out where that pixel “lands” in the source image. Since the scale factor is rarely a whole number, that landing spot is almost always between four source pixels — so mapper.v walks an accumulator (fixed-point, 8 fractional bits by default) across the source image at the right rate and spits out: which source row/column pair is above-left of the landing spot, which is below-right, and exactly how far into that gap (frac_x, frac_y) the point sits. It also tags each row with a

mod-3 label that stage 2 uses to know which cache slot that row lives in — more on that below.

Stage 2 — `buffer.v`

This is the stage that actually deals with “we can't keep the whole image.” Because bilinear scaling only ever needs two source rows at a time (the one above the landing point and the one below), `buffer.v` only keeps three rows resident at once, in a rotating cache — row N stored in slot $(N \bmod 3)$. Each slot is further split into two BRAMs (even columns / odd columns) so it can read out two neighbouring pixels from the same row in a single cycle. Once stage 1 has moved on far enough that a slot's row can no longer be needed, that slot is free to be overwritten by the writer side, which is quietly streaming the source image in from the AXI-Stream input in the background. This is a real producer/consumer relationship with backpressure in both directions — the source only advances when there's room, and stage 1 only advances when the rows it needs are actually the rows sitting in the cache (not just “some row, hopefully the right one”).

There's also a simpler, non-BRAM version of this stage further down in the same file (kept as an alternate implementation) that just holds the entire source frame in one big array. It's there for quick testing or for source images small enough that the memory cost doesn't matter — swap it in if you don't need the rolling-cache behaviour.

Stage 3 — `horizontal_interpolator.v`

Takes the four neighbour pixels (TL, TR, BL, BR) and does the horizontal half of the blend: it interpolates along the top row (between TL and TR) and along the bottom row (between BL and BR), using `frac_x` as the blend weight. Output is two partially-blended pixels, handed to stage 4 along with `frac_y`.

Stage 4 — `vertical_interpolator.v`

Takes those two horizontally-blended values and finishes the job, blending between them using `frac_y` — the vertical position of the landing point. The result is pushed out over `m_axis_tdata/tvalid/tlast`, with `tlast` marking the end of every destination row so a downstream consumer knows where rows end without needing to count pixels itself.

Every stage is parameterized by `NUM_CH` and `CH_W`, so the same pipeline handles single-channel grayscale (`NUM_CH=1`, `CH_W=8`) or packed 24-bit RGB (`NUM_CH=3`, `CH_W=8`) without any code changes — just a different `tdata` width.

Challenges faced and how they were resolved

Building a stateful streaming pipeline in hardware surfaces a different class of problem than a simple combinational design does: correctness has to hold up across an unbounded sequence of pixels, not just for a handful of test vectors. A few issues came up during development that were instructive enough to be worth documenting here, along with how each was diagnosed and fixed.

Keeping the row cache synchronized under non-integer scale ratios

The row cache identifies which of its three slots a given source row belongs in using a modulo-3 tag. Early on, this tag was updated incrementally on each destination row, which is only valid when every

destination row maps to the very next source row in sequence. Once downscaling was introduced — where the mapper deliberately skips source rows to keep pace — that assumption no longer held, and the tag could drift out of sync with the row actually occupying a given slot. The resolution was to derive the tag directly from the destination row's true source-row mapping on every cycle, rather than stepping it incrementally, which holds correctly for any scale ratio.

Distinguishing “slot has data” from “slot has the right data”

The buffer stage originally gated reads purely on whether a cache slot had been written at all. Under downscaling, however, more than one source row can pass through the same slot before that data is consumed, so a simple written/not-written flag isn't sufficient to guarantee correctness. The fix was to track the specific source row number held in each slot and check that identity explicitly before permitting a read, rather than relying on occupancy alone.

A dropped final pixel at the end of every frame

The coordinate mapper occasionally failed to emit the very last pixel of each frame. The root cause was two writes to the same valid signal within the same clock edge, where a later statement intended as a safeguard was silently overriding the intended value for that final pixel. The fix was to remove the redundant write and let the state machine's idle transition clear the signal on the following cycle instead.

Row-boundary detection for image widths that aren't powers of two

The source-image reader initially detected the end of a row by inspecting a fixed-width slice of the linear pixel address. This works correctly only when the image width is a power of two; for other widths (such as 480 or 451 pixels), the address slice wraps at a different point than the actual row boundary, causing the row-end marker to drift progressively further off with each row. This was resolved by introducing a dedicated column counter that wraps exactly at the configured image width, independent of the address encoding.

In each case, the testbench's row-level logging — tracking write completions, row-done events, and cache-validity transitions — was what made these issues traceable to a specific stage and condition rather than requiring broader guesswork.

Handshaking, briefly

Every stage boundary in this design uses the same valid/ready contract AXI-Stream uses at the edges: the sender asserts valid and holds its data steady until the receiver asserts ready on the same cycle — only then has the transfer actually happened. Readiness is generally computed backwards from the end of the pipeline: stage 4 is ready whenever its output register is free or the downstream consumer is pulling data (`m_axis_tready`), and that readiness ripples back through stage 3, stage 2, and stage 1 as each one's own `s*_ready`. It means the whole pipeline can stall cleanly from a single slow consumer at the far end, with nothing overwritten and nothing dropped.

What's in this repo

File	What it's for
image_scaler_top.v	Top module. Wires stage1→stage2→stage3→stage4 together.
mapper.v	Stage 1 — destination→source coordinate mapping and fractional weights.
buffer.v	Stage 2 — rolling 3-row source cache and 4-neighbour pixel fetch.
bram.v	Single dual-port-style memory block used by buffer.v's cache slots.
horizontal_interpolator.v	Stage 3 — blends TL/TR and BL/BR along X.
vertical_interpolator.v	Stage 4 — blends the two row results along Y, drives the output stream.
input_image.v	Simulation-only AXI-Stream source: preloads a frame from input.hex and streams it out.
output_image.v	Simulation-only AXI-Stream sink: captures the scaled frame into output.hex.
tb_image_scaler.v	Testbench: instantiates source → scaler → sink, drives reset/start, logs progress, dumps output.hex.
png_to_hex.py	Converts a PNG into the .hex format \$readmemh expects (grayscale or packed RGB).
hex_to_png.py	Converts the scaler's output .hex back into a viewable PNG.
run	The whole flow in four commands: image→hex, compile, simulate, hex→image.

Running it yourself

Everything is driven from icarus verilog (iverilog/vvp) plus two small Python helpers that get an image in and out of the hex format Verilog's \$readmemh / \$writememh expect. There's no vendor tooling required.

```
# 1. turn your source PNG into a hex file the testbench can $readmemh
python3 png_to_hex.py image.png input.hex # grayscale
python3 png_to_hex.py image.png input.hex --rgb # 24-bit RGB

# 2. compile the whole design + testbench
iverilog -g2012 -o scaler_sim tb_image_scaler.v image_scaler_top.v \
mapper.v buffer.v horizontal_interpolator.v vertical_interpolator.v \
input_image.v output_image.v bram.v

# 3. run the simulation – writes output.hex once the full frame is out
```

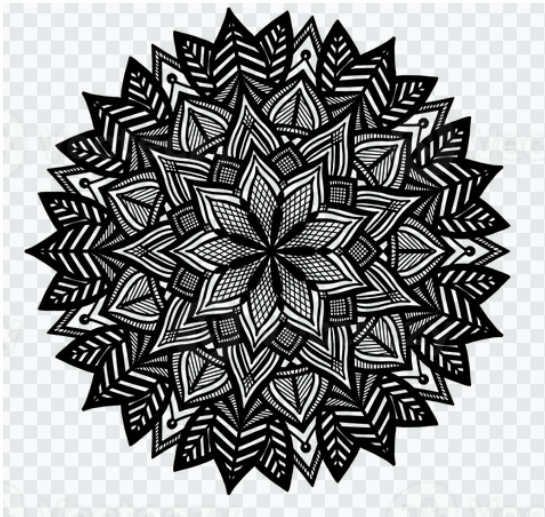
```
vvp scaler_sim
```

```
# 4. turn the result back into a PNG you can actually look at  
python3 hex_to_png.py output.hex result.png --width 960 --height 500  
python3 hex_to_png.py output.hex result.png --width 960 --height 500 --rgb
```

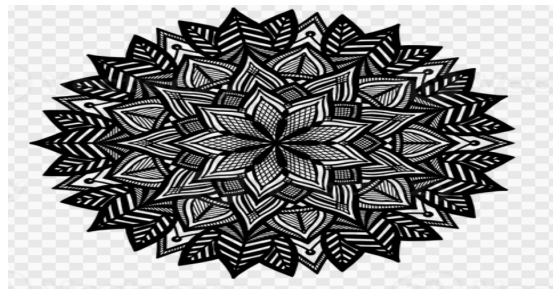
SRC_W / SRC_H / DST_W / DST_H are set as localparams at the top of tb_image_scaler.v — change those (and the matching --width/--height flags above) to try a different image or scale factor. The testbench will run until it's collected a full destination frame, print sample pixel values, and dump waves.vcd if you want to look at the handshakes directly.

A real run

As an example: a 480×451 source image scaled to 960×500 — doubling the width while barely touching the height. That's a deliberately lopsided scale factor (X and Y move at very different rates) specifically because it's a good stress test for the mapper/buffer handshake: X and Y fractional accumulators are advancing at completely different speeds, which is exactly the situation the mod-3 desync bug above used to break under.



source — 480 × 451



scaler output — 960 × 500

The stretch is intentional and the interpolation is doing its job — note there's no blockiness or ghosting at the edges of the design, which is the thing that would show up first if the neighbour-fetch or fractional weighting were wrong.

This project reflects an ongoing effort to build hardware designs that hold up under realistic streaming conditions, not just idealized test cases. Feedback and contributions are welcome.